ASP.NET Core CRUD Application with AJAX - Complete Documentation (Hinglish)

Table of Contents (Vishay Suchi)

Project Overview (Project Ka Overview)

Yeh project kya hai?

Yeh ek **User Registration System** hai jo ASP.NET Core MVC se banaya gaya hai. Isme aap yeh kar sakte ho:

- Countries ko manage karo (Create, Read, Update, Delete)
- States ko manage karo (Create, Read, Update, Delete)
- Users ko register karo Country aur State selection ke saath
- Saare operations bina page refresh ke AJAX se hote hain

Technologies Jo Use Hui Hain

- **ASP.NET Core 8.0** - Web framework
- **Entity Framework Core** - Database ORM (Object-Relational Mapping)
- **PostgreSQL** - Database
- **jQuery & AJAX** - Frontend interactivity
- **Bootstrap** - UI styling

Key Features (Mukhya Visheshatayen)

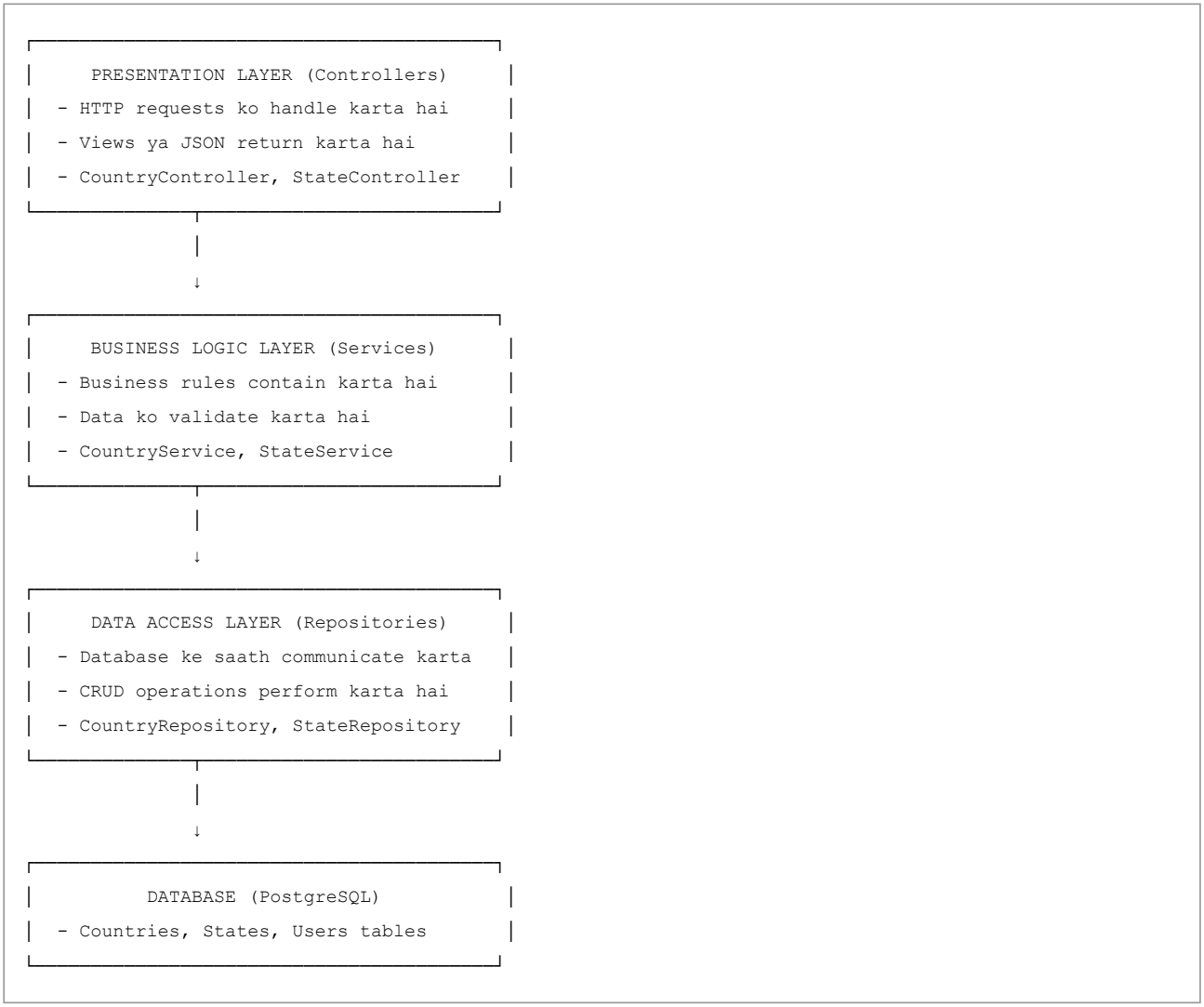
1. **Three-Layer Architecture** - Har layer ki apni responsibility
2. **AJAX-based CRUD** - Page reload nahi hota
3. **Cascading Dropdowns** - State dropdown Country selection ke basis par change hota hai
4. **Soft Delete** - Records permanently delete nahi hote, sirf hide hote hain
5. **Dependency Injection** - Components ke beech loose coupling

---

Architecture & Design Patterns (Architecture Aur Design Patterns)

Three-Layer Architecture (Teen Layer Ka Architecture)

Is project mein **three-layer architecture** pattern use hua hai:



Teen Layers Kyun?

Fayde:

1. **Separation of Concerns** - Har layer ki apni specific responsibility hai
2. **Maintainability** - Ek layer ko modify karna easy hai bina dusre ko affect kiye
3. **Testability** - Har layer ko independently test kar sakte hain
4. **Reusability** - Business logic ko different controllers mein reuse kar sakte hain

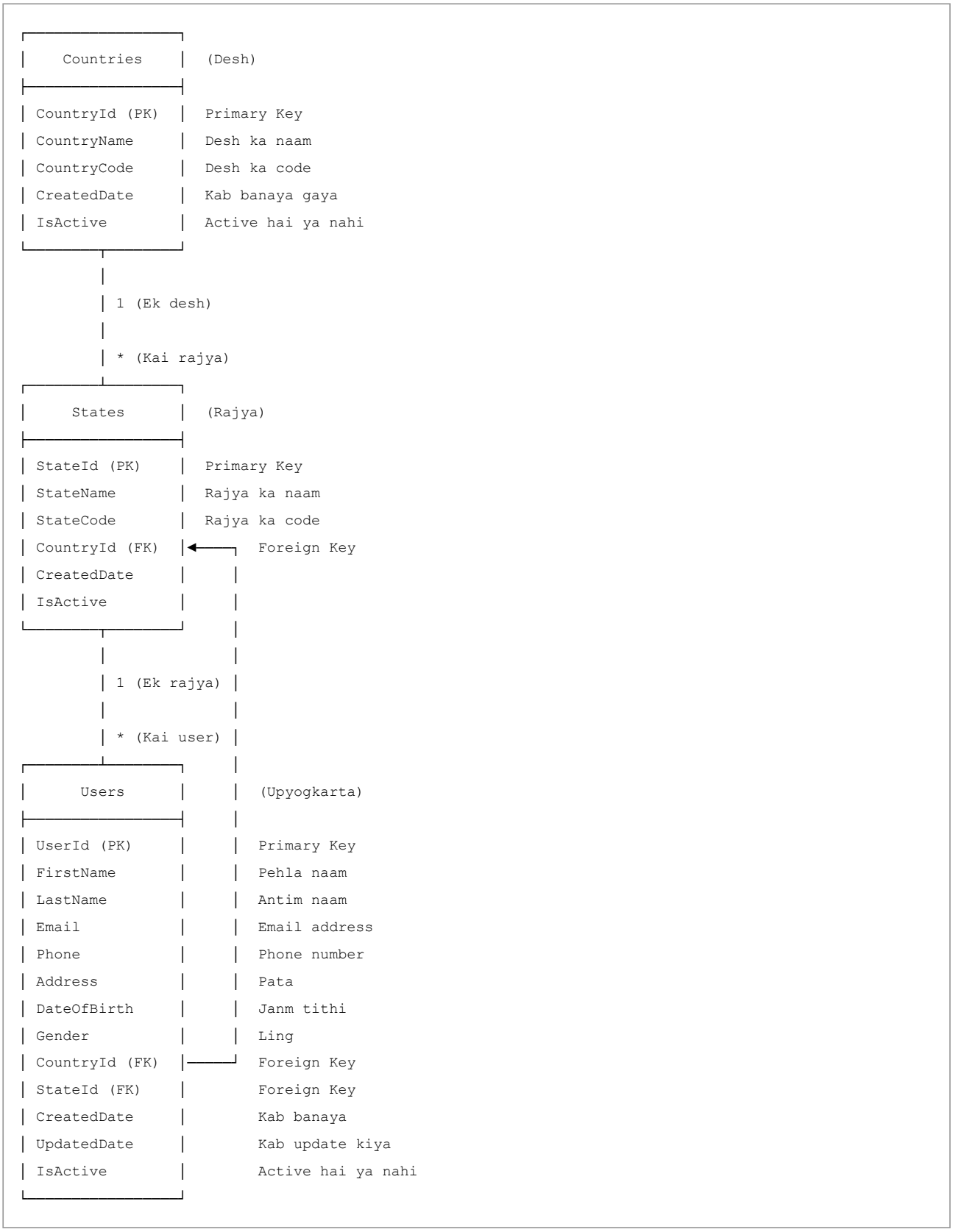
---

Project Structure (Project Ki Structure)

```
Practicecrudwithajax/
├── Model/                          # Data Models (Entities)
│   ├── CountryModel/
│   │   └── Country.cs              # Country entity - Desh ka model
│   ├── StateModel/
│   │   └── State.cs                # State entity - Rajya ka model
│   └── UserModel/
│       └── User.cs                 # User entity - User ka model
│
├── Data/                           # Database Context
│   ├── ApplicationDbContext.cs     # EF Core DbContext - Database connection
│   └── Migrations/                 # Database migrations - Database changes
│
├── Dal/                            # Data Access Layer (Repositories)
│   ├── CountryDAL/
│   │   ├── ICountryRepository.cs # Interface - Contract
│   │   └── CountryRepository.cs  # Implementation - Actual code
│   ├── StateDAL/
│   │   ├── IStateRepository.cs
│   │   └── StateRepository.cs
│   └── UserDAL/
│       ├── IUserRepository.cs
│       └── UserRepository.cs
│
├── Bal/                            # Business Logic Layer (Services)
│   ├── CountryBAL/
│   │   ├── ICountryService.cs    # Interface
│   │   └── CountryService.cs     # Implementation
│   ├── StateBAL/
│   │   ├── IStateService.cs
│   │   └── StateService.cs
│   └── UserBAL/
│       ├── IUserService.cs
│       └── UserService.cs
│
└── Practicecrudwithajax/           # Web Application
    ├── Controllers/                # MVC Controllers - Request handlers
    │   ├── CountryController.cs
    │   ├── StateController.cs
    │   └── UserController.cs
    ├── Views/                      # Razor Views - HTML pages
    │   ├── Country/
    │   │   └── Index.cshtml
    │   ├── State/
    │   │   └── Index.cshtml
    │   └── User/
    │       └── Index.cshtml
    ├── wwwroot/                    # Static files (CSS, JS, images)
    ├── Program.cs                  # Application entry point - Shuruat
    └── appsettings.json            # Configuration - Settings
```

Database Design (Database Ka Design)

Entity Relationship Diagram (ERD)



Relationships (Rishte)

1. **Country → States** (One-to-Many / Ek-se-Kai)
    - Ek Country mein kai States ho sakte hain
    - Har State ek hi Country se belong karta hai
    - Example: India mein Maharashtra, Delhi, Gujarat, etc.
2. **Country → Users** (One-to-Many / Ek-se-Kai)
    - Ek Country mein kai Users ho sakte hain
    - Har User ek hi Country se belong karta hai
    - Example: India se kai users register ho sakte hain
3. **State → Users** (One-to-Many / Ek-se-Kai)
    - Ek State mein kai Users ho sakte hain
    - Har User ek hi State se belong karta hai
    - Example: Maharashtra se kai users register ho sakte hain

Soft Delete Strategy (Soft Delete Ki Strategy)

Records ko permanently delete karne ke bajaye, hum **soft delete** use karte hain:

- Har table mein `IsActive` field hai (boolean - true/false)
- "Delete" karte waqt, hum `IsActive = false` set karte hain
- Queries mein `IsActive = true` se filter karte hain taaki sirf active records dikhen

Fayde:

- Data recovery possible hai
- Referential integrity maintain rehti hai
- Audit trail preserved rehta hai
- Galti se delete kiya toh wapas la sakte hain

Layer-by-Layer Explanation (Har Layer Ki Vyakhya)

1. Model Layer (Entities) - Data Ka Structure

Purpose (Uddeshya): Data ka structure define karna

Example: Country.cs

```
public class Country
{
    public int CountryId { get; set; }          // Primary Key - Unique ID
    public string CountryName { get; set; }     // Required field - Zaroori field
    public string? CountryCode { get; set; }    // Optional field - Optional
    public DateTime CreatedDate { get; set; }   // Kab banaya gaya
    public bool IsActive { get; set; }          // Active hai ya nahi

    // Navigation Properties (Relationships) - Dusre tables se connection
    public virtual ICollection<State>? States { get; set; }
    public virtual ICollection<User>? Users { get; set; }
}
```

Key Concepts:

- **Data Annotations** - `[Required]`, `[StringLength]`, etc. validation ke liye
- **Navigation Properties** - Entities ke beech relationships define karte hain
- **Virtual keyword** - Entity Framework mein lazy loading enable karta hai

2. Data Layer (ApplicationDbContext) - Database Connection

Purpose: Application aur database ke beech ka pul

Example: ApplicationDbContext.cs

```
public class ApplicationDbContext : DbContext
{
    // DbSet = Database mein table
    public DbSet<Country> Countries { get; set; }
    public DbSet<State> States { get; set; }
    public DbSet<User> Users { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Relationships configure karna
        modelBuilder.Entity<Country>()
            .HasMany(c => c.States)           // Ek country ke kai states
            .WithOne(s => s.Country)          // Har state ka ek country
            .HasForeignKey(s => s.CountryId); // Foreign key

        // Soft delete ke liye global filter
        modelBuilder.Entity<Country>()
            .HasQueryFilter(c => c.IsActive);  // Sirf active records
    }
}
```

Key Concepts:

- **DbSet** - Database mein ek table ko represent karta hai
- **OnModelCreating** - Entity relationships aur constraints configure karte hain
- **HasQueryFilter** - Automatically soft-deleted records ko filter kar deta hai

3. Repository Layer (Data Access) - Database Operations

Purpose: Database operations perform karna (CRUD)

Example: CountryRepository.cs

```
public class CountryRepository : ICountryRepository
{
    private readonly ApplicationDbContext _context;

    // Constructor - Dependency Injection se context milta hai
    public CountryRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    // Sabhi active countries ko fetch karna
    public async Task<List<Country>> GetAllCountriesAsync()
    {
        // LINQ Query:
        // 1. Countries table se data lo
        // 2. Sirf active countries filter karo
        // 3. Name se sort karo
        // 4. List mein convert karo

        return await _context.Countries
            .Where(c => c.IsActive)           // Filter: sirf active
            .OrderBy(c => c.CountryName)      // Sort: naam se
            .ToListAsync();                   // Execute query
    }

    // Naya country add karna
    public async Task<Country> AddCountryAsync(Country country)
    {
        // Step 1: Creation date aur active status set karo
        country.CreatedDate = DateTime.UtcNow;
        country.IsActive = true;

        // Step 2: DbSet mein add karo (memory mein)
        await _context.Countries.AddAsync(country);

        // Step 3: Database mein save karo (actual INSERT)
        await _context.SaveChangesAsync();

        // Step 4: Added country return karo (ab ID bhi hai)
        return country;
    }

    // Country ko soft delete karna
    public async Task<bool> DeleteCountryAsync(int id)
    {
        // Step 1: Country dhundo
        var country = await _context.Countries
            .FirstOrDefaultAsync(c => c.CountryId == id);

        // Step 2: Agar nahi mila toh false return karo
        if (country == null)
            return false;
```

```
        // Step 3: Soft delete - IsActive = false
        // IMPORTANT: Record delete nahi hota, sirf hide hota hai
        country.IsActive = false;

        // Step 4: Database mein save karo
        await _context.SaveChangesAsync();

        // Step 5: Success - true return karo
        return true;
    }
}
```

Key Concepts:

- **LINQ Queries** - `.Where()`, `.OrderBy()`, `.FirstOrDefaultAsync()` - Data filter aur sort karne ke liye
- **Async/Await** - Database operations non-blocking hote hain
- **SaveChangesAsync()** - Changes ko database mein commit karta hai

4. Service Layer (Business Logic) - Business Rules

Purpose: Business rules aur validation

Example: CountryService.cs

```
public class CountryService : ICountryService
{
    private readonly ICountryRepository _repository;

    // Constructor - Repository inject hota hai
    public CountryService(ICountryRepository repository)
    {
        _repository = repository;
    }

    public async Task<List<Country>> GetAllCountriesAsync()
    {
        // Yahan business logic add kar sakte hain
        // Example: Operation log karna, permissions check karna, etc.
        return await _repository.GetAllCountriesAsync();
    }

    public async Task<Country> AddCountryAsync(Country country)
    {
        // Business validation yahan
        // Example: Check karo ki country name already exist toh nahi karta

        // Agar sab theek hai toh repository ko call karo
        return await _repository.AddCountryAsync(country);
    }
}
```

Key Concepts:

- **Business Validation** - Application-specific rules
- **Abstraction** - Service ko database details nahi pata
- **Single Responsibility** - Har service ek entity handle karta hai

5. Controller Layer (Presentation) - Request Handling

Purpose: HTTP requests ko handle karna aur responses return karna

Example: CountryController.cs

```
public class CountryController : Controller
{
    private readonly ICountryService _countryService;

    // Constructor - Service inject hota hai
    public CountryController(ICountryService countryService)
    {
        _countryService = countryService;
    }

    // View return karta hai
    public IActionResult Index()
    {
        return View();  // Views/Country/Index.cshtml
    }

    // AJAX endpoint - JSON return karta hai
    [HttpGet]
    public async Task<JsonResult> GetAllCountries()
    {
        var countries = await _countryService.GetAllCountriesAsync();
        return Json(countries);
    }

    // AJAX endpoint - Country create karta hai
    [HttpPost]
    public async Task<JsonResult> Create(Country country)
    {
        // Validation check karo
        if (ModelState.IsValid)
        {
            await _countryService.AddCountryAsync(country);
            return Json(new { success = true, message = "Country add ho gaya!" });
        }
        return Json(new { success = false, message = "Invalid data" });
    }
}
```

Key Concepts:

- **Action Methods** - Methods jo requests handle karte hain

- **HTTP Verbs** - `[HttpGet]`, `[HttpPost]` - Request type specify karte hain
- **JsonResult** - AJAX calls ke liye JSON return karta hai
- **ModelState** - Model ki validation state

Controllers Explained (Controllers Ki Detailed Vyakhya)

Controllers Kyun Zaroori Hain?

Controllers MVC application mein sabhi HTTP requests ka **entry point** hain.

Flow:

1. User button click karta hai ya form submit karta hai
2. Browser HTTP request server ko bhejta hai
3. **Controller** request receive karta hai
4. Controller **Service** ko call karta hai business logic ke liye
5. Service **Repository** ko call karta hai data ke liye
6. Data wapas aata hai: Repository → Service → Controller
7. Controller **View** ya **JSON** browser ko return karta hai

CountryController - Detailed Explanation

Purpose: Country CRUD operations manage karna

Methods:

1. **Index()** - Main view return karta hai

```
public IActionResult Index()
{
    return View();  // Views/Country/Index.cshtml return karta hai
}
```

- **Kyun?** Country management page dikhane ke liye
- **Kab call hota hai?** Jab user /Country/Index URL par jata hai
- **Kya return karta hai?** HTML view

2. **GetAllCountries()** - Sabhi countries fetch karta hai

```
[HttpGet]
public async Task<JsonResult> GetAllCountries()
{
    var countries = await _countryService.GetAllCountriesAsync();
    return Json(countries);
}
```

- **Kyun?** Page par countries table populate karne ke liye
- **Kab call hota hai?** Jab page load hota hai ya add/update/delete ke baad
- **Kya return karta hai?** JSON array of countries

3. **Create(Country country)** - Naya country add karta hai

```
[HttpPost]
public async Task<JsonResult> Create(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.AddCountryAsync(country);
        return Json(new { success = true, message = "Country add ho gaya!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

- **Kyun?** Database mein naya country add karne ke liye
- **Kab call hota hai?** Jab user "Add Country" form submit karta hai
- **Kya return karta hai?** JSON with success status aur message

4. **GetById(int id)** - Ek country fetch karta hai

```
[HttpGet]
public async Task<JsonResult> GetById(int id)
{
    var country = await _countryService.GetCountryByIdAsync(id);
    return Json(country);
}
```

- **Kyun?** Edit form ko existing data se populate karne ke liye
- **Kab call hota hai?** Jab user "Edit" button click karta hai
- **Kya return karta hai?** JSON object of the country

5. **Update(Country country)** - Country update karta hai

```
[HttpPost]
public async Task<JsonResult> Update(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.UpdateCountryAsync(country);
        return Json(new { success = true, message = "Country update ho gaya!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

- **Kyun?** Existing country mein changes save karne ke liye
- **Kab call hota hai?** Jab user "Edit Country" form submit karta hai
- **Kya return karta hai?** JSON with success status aur message

6. **Delete(int id)** - Country delete karta hai

```
[HttpPost]
public async Task<JsonResult> Delete(int id)
{
    var result = await _countryService.DeleteCountryAsync(id);
    if (result)
    {
        return Json(new { success = true, message = "Country delete ho gaya!" });
    }
    return Json(new { success = false, message = "Error deleting" });
}
```

- **Kyun?** Country ko soft delete karne ke liye (IsActive = false)
- **Kab call hota hai?** Jab user "Delete" button click karta hai aur confirm karta hai
- **Kya return karta hai?** JSON with success status aur message

StateController - Special Features

Cascading Dropdown Implementation:

```
[HttpGet]
public async Task<JsonResult> GetStatesByCountry(int countryId)
{
    var states = await _stateService.GetStatesByCountryIdAsync(countryId);
    var result = states.Select(s => new
    {
        s.StateId,
        s.StateName
    });
    return Json(result);
}
```

Yeh kyun important hai?

- Jab user Country select karta hai, State dropdown mein sirf us country ke states dikne chahiye
- Yeh method CountryId se states ko filter karta hai
- AJAX se call hota hai jab Country dropdown change hota hai

Example Flow:

1. User "India" select karta hai Country dropdown se
2. JavaScript change detect karta hai
3. AJAX call hota hai `/State/GetStatesByCountry?countryId=1`
4. Sirf Indian states return hote hain
5. State dropdown Indian states se populate hota hai (Maharashtra, Delhi, etc.)

UserController - Complete Registration

Purpose: User registration handle karna Country aur State selection ke saath

Key Method:

```
[HttpPost]
public async Task<JsonResult> Create(User user)
{
    if (ModelState.IsValid)
    {
        await _userService.AddUserAsync(user);
        return Json(new { success = true, message = "User register ho gaya!" });
    }

    // Validation errors collect karo
    var errors = ModelState.Values
        .SelectMany(v => v.Errors)
        .Select(e => e.ErrorMessage);
    return Json(new { success = false, message = "Validation fail", errors });
}
```

Validation:

- Email format validation
- Required field validation
- Phone number format validation
- Sab User model mein Data Annotations se define hai

AJAX Implementation (AJAX Ka Implementation)

AJAX Kya Hai?

AJAX = Asynchronous JavaScript and XML

Fayde:

- Page reload kiye bina page ke parts update kar sakte hain
- Better user experience
- Faster interactions
- Smooth aur responsive feel

Is Project Mein AJAX Flow

```
User Action (Button click)
    ↓
JavaScript Event Handler
    ↓
$.ajax() call server ko
    ↓
Controller request receive karta hai
    ↓
Service request process karta hai
    ↓
Repository database access karta hai
    ↓
Data wapas Controller ko aata hai
    ↓
Controller JSON return karta hai
    ↓
AJAX success callback
    ↓
Page ko naye data se update karo
```

Example: AJAX Se Country Add Karna

Frontend (JavaScript):

```
function addCountry() {
    // Form se data collect karo
    var countryData = {
        CountryName: $('#countryName').val(),
        CountryCode: $('#countryCode').val()
    };

    // AJAX call
    $.ajax({
        url: '/Country/Create',        // Controller method ka URL
        type: 'POST',                  // HTTP method
        data: countryData,             // Bhejne wala data
        success: function(response) {  // Success callback
            if (response.success) {
                alert(response.message);
                loadCountries();       // Table refresh karo
                $('#addForm').trigger('reset');  // Form clear karo
            } else {
                alert(response.message);
            }
        },
        error: function() {            // Error callback
            alert('Error occurred');
        }
    });
}
```

Backend (Controller):

```
[HttpPost]
public async Task<JsonResult> Create(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.AddCountryAsync(country);
        return Json(new { success = true, message = "Country add ho gaya!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

Key Points:

1. JavaScript form data collect karta hai
2. AJAX POST request `/Country/Create` ko bhejta hai
3. Controller data validate aur save karta hai
4. Controller JSON response return karta hai
5. JavaScript response receive karta hai aur UI update karta hai

---

Setup & Installation (Setup Aur Installation)

Prerequisites (Zaroori Cheezein)

1. **.NET 8.0 SDK** - microsoft.com/dotnet se download karo
2. **PostgreSQL** - postgresql.org se download karo
3. **Visual Studio 2022** ya **VS Code**
4. **Git** (optional)

Step-by-Step Setup (Kadam-dar-Kadam Setup)

Step 1: Project Clone Ya Download Karo

```
git clone <repository-url>
cd Practicecrudwithajax
```

Step 2: Database Connection Configure Karo

`appsettings.json` file edit karo:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Database=PracticeCrudDb;Username=postgres;Password=yourpassword"
  }
}
```

Replace karo:

- `localhost` - Aapka PostgreSQL server address

- `PracticeCrudDb` - Aapka database name
- `postgres` - Aapka PostgreSQL username
- `yourpassword` - Aapka PostgreSQL password

Step 3: NuGet Packages Restore Karo

```
dotnet restore
```

Step 4: Database Create Karo

Database automatically create ho jayega jab aap migrations run karoge.

Step 5: Migrations Apply Karo

```
cd Data
dotnet ef database update
```

Yeh command aapke PostgreSQL database mein saari tables create kar dega.

Step 6: Application Run Karo

```
cd ../Practicecrudwithajax
dotnet run
```

Step 7: Browser Mein Kholo

```
https://localhost:5001
```

Troubleshooting (Agar Problem Aaye)

Problem: "Connection refused" error **Solution:** Check karo ki PostgreSQL chal raha hai ya nahi

Problem: "Migration not found" **Solution:** Pehle `dotnet ef migrations add InitialCreate` run karo

Problem: "Port already in use" **Solution:** `launchSettings.json` mein port change karo

---

API Endpoints Reference (API Endpoints Ki List)

Country Endpoints

Method	Endpoint	Purpose	Parameters	Returns
GET	`/Country/Index`	Page display karna	None	HTML View
GET	`/Country/GetAllCountries`	Sabhi countries	None	JSON array
GET	`/Country/GetById?id={id}`	Ek country	id (int)	JSON object
POST	`/Country/Create`	Country add karna	Country object	JSON response
POST	`/Country/Update`	Country update karna	Country object	JSON response
POST	`/Country/Delete?id={id}`	Country delete karna	id (int)	JSON response

State Endpoints

Method	Endpoint	Purpose	Parameters	Returns

Method	Endpoint	Purpose	Parameters	Returns
GET	`/State/Index`	Page display karna	None	HTML View
GET	`/State/GetAllStates`	Sabhi states	None	JSON array
GET	`/State/GetStatesByCountry?countryId={id}`	Country ke states	countryId (int)	JSON array
GET	`/State/GetById?id={id}`	Ek state	id (int)	JSON object
POST	`/State/Create`	State add karna	State object	JSON response
POST	`/State/Update`	State update karna	State object	JSON response
POST	`/State/Delete?id={id}`	State delete karna	id (int)	JSON response

User Endpoints

Method	Endpoint	Purpose	Parameters	Returns
GET	`/User/Index`	Page display karna	None	HTML View
GET	`/User/GetAllUsers`	Sabhi users	None	JSON array
GET	`/User/GetById?id={id}`	Ek user	id (int)	JSON object
POST	`/User/Create`	User register karna	User object	JSON response
POST	`/User/Update`	User update karna	User object	JSON response
POST	`/User/Delete?id={id}`	User delete karna	id (int)	JSON response

Common Concepts for Beginners (Beginners Ke Liye Common Concepts)

1. Dependency Injection Kya Hai?

Simple Explanation: Objects khud banane ke bajaye, aap framework se maangte ho ki woh provide kare.

Bina DI ke:

```
public class CountryController
{
    private ICountryService _service;

    public CountryController()
    {
        _service = new CountryService();  // Manually create kar rahe hain
    }
}
```

DI ke saath:

```
public class CountryController
{
    private ICountryService _service;

    public CountryController(ICountryService service)
    {
        _service = service;  // Framework provide karta hai
    }
}
```

Fayde:

- Testing easy ho jati hai (mock objects inject kar sakte hain)
- Loose coupling (controller ko implementation nahi pata)
- Centralized configuration (Program.cs mein)

2. Async/Await Kya Hai?

Purpose: Database wait karte waqt application ko block mat karo

Synchronous (Blocking):

```
public List<Country> GetAllCountries()
{
    // Application yahan wait karti hai, kuch aur nahi kar sakti
    return _context.Countries.ToList();
}
```

Asynchronous (Non-blocking):

```
public async Task<List<Country>> GetAllCountriesAsync()
{
    // Application wait karte waqt dusre requests handle kar sakti hai
    return await _context.Countries.ToListAsync();
}
```

Fayde:

- Better performance
- Zyada concurrent users handle kar sakte hain
- Application responsive rehti hai

3. LINQ Kya Hai?

LINQ = Language Integrated Query

Purpose: C# syntax use karke collections ko query karna

Example:

```
// Sabhi active countries, naam se sorted
var countries = _context.Countries
    .Where(c => c.IsActive)           // Filter
    .OrderBy(c => c.CountryName)      // Sort
    .ToList();                        // Execute

// Yeh SQL generate karta hai:
// SELECT * FROM Countries WHERE IsActive = true ORDER BY CountryName
```

Common LINQ Methods:

- `.Where()` - Filter karna
- `.OrderBy()` - Ascending sort

- `.OrderByDescending()` - Descending sort
- `.Select()` - Transform/project karna
- `.FirstOrDefault()` - Pehla ya null
- `.Any()` - Koi exist karta hai ya nahi
- `.Count()` - Records count karna

4. Entity Framework Core Kya Hai?

EF Core = Object-Relational Mapper (ORM)

Purpose: SQL ke bajaye C# objects use karke database ke saath kaam karna

Bina EF Core ke (Raw SQL):

```
string sql = "INSERT INTO Countries (CountryName, IsActive) VALUES (@name, @active)";
// SQL command execute karna...
```

EF Core ke saath:

```
var country = new Country { CountryName = "India", IsActive = true };
_context.Countries.Add(country);
_context.SaveChanges();
```

Fayde:

- Type-safe (compiler errors check karta hai)
- Kam code
- Database-agnostic (database easily switch kar sakte hain)
- Automatic SQL generation

5. Model Validation Kya Hai?

Purpose: Save karne se pehle ensure karna ki data requirements meet karta hai

Data Annotations:

```
public class Country
{
    [Required(ErrorMessage = "Country name zaroori hai")]
    [StringLength(100, ErrorMessage = "Max 100 characters")]
    public string CountryName { get; set; }

    [EmailAddress(ErrorMessage = "Invalid email format")]
    public string Email { get; set; }
}
```

Controller Mein Validation:

```
if (ModelState.IsValid)
{
    // Data valid hai, save karo
}
else
{
    // Data invalid hai, errors return karo
}
```

6. Soft Delete Kya Hai?

Hard Delete: Record ko database se permanently remove karna

```
DELETE FROM Countries WHERE CountryId = 1
```

Soft Delete: Record ko inactive mark karna

```
UPDATE Countries SET IsActive = false WHERE CountryId = 1
```

Fayde:

- Deleted data recover kar sakte hain
- Referential integrity maintain rehti hai
- Audit trail preserved rehta hai
- Galti se delete kiya toh wapas la sakte hain

Implementation:

```
public async Task<bool> DeleteCountryAsync(int id)
{
    var country = await _context.Countries.FindAsync(id);
    if (country == null) return false;

    country.IsActive = false;  // Soft delete
    await _context.SaveChangesAsync();
    return true;
}
```

7. Cascading Dropdown Kya Hai?

Purpose: Dusra dropdown pehle dropdown ki selection par depend karta hai

Example:

1. User "India" select karta hai Country dropdown se
2. State dropdown mein sirf Indian states dikhte hain (Maharashtra, Delhi, etc.)
3. User "USA" select karta hai Country dropdown se
4. State dropdown change ho jata hai aur sirf US states dikhte hain (California, Texas, etc.)

Implementation:

```
$('#countryDropdown').change(function() {
    var countryId = $(this).val();

    $.ajax({
        url: '/State/GetStatesByCountry',
        data: { countryId: countryId },
        success: function(states) {
            $('#stateDropdown').empty();
            $.each(states, function(i, state) {
                $('#stateDropdown').append(
                    $('<option>').val(state.stateId).text(state.stateName)
                );
            });
        }
    });
});
```

Conclusion (Nishkarsh)

Yeh documentation ASP.NET Core CRUD application with AJAX ke sabhi aspects ko cover karta hai. Is project mein yeh demonstrate kiya gaya hai:

1. **Clean Architecture** - Teen layers ka separation
2. **Best Practices** - Dependency Injection, Async/Await, Repository Pattern
3. **Modern Web Development** - AJAX, RESTful APIs, Responsive UI
4. **Database Design** - Relationships, Soft Delete, Migrations

Aage Seekhne Ke Liye (Next Steps for Learning)

1. **Authentication Add Karo** - User login/logout
2. **Authorization Add Karo** - Role-based access control
3. **Logging Add Karo** - Errors aur operations track karo
4. **Unit Tests Add Karo** - Apne code ko test karo
5. **API Documentation Add Karo** - Swagger/OpenAPI
6. **Cloud Par Deploy Karo** - Azure, AWS, ya dusre platforms

Resources (Saadhan)

- **Official Docs:** https://docs.microsoft.com/aspnet/core (https://docs.microsoft.com/aspnet/core)
- **Entity Framework:** https://docs.microsoft.com/ef/core (https://docs.microsoft.com/ef/core)
- **C# Guide:** https://docs.microsoft.com/dotnet/csharp (https://docs.microsoft.com/dotnet/csharp)
- **PostgreSQL:** https://www.postgresql.org/docs (https://www.postgresql.org/docs)

Document Version: 1.0

Last Updated: January 2026

Author: Practice CRUD with AJAX Project

Note: Yeh documentation beginners ke liye banaya gaya hai. Har concept ko simple language mein explain kiya gaya hai with examples.

ASP.NET Core CRUD Application with AJAX - Complete Documentation

Table of Contents

Project Overview

What is this project?

This is a **User Registration System** built with ASP.NET Core MVC that allows you to:

- Manage Countries (Create, Read, Update, Delete)
- Manage States (Create, Read, Update, Delete)
- Register Users with Country and State selection
- All operations happen without page refresh using AJAX

Technologies Used

- **ASP.NET Core 8.0** - Web framework
- **Entity Framework Core** - Database ORM (Object-Relational Mapping)
- **PostgreSQL** - Database
- **jQuery & AJAX** - Frontend interactivity
- **Bootstrap** - UI styling

Key Features

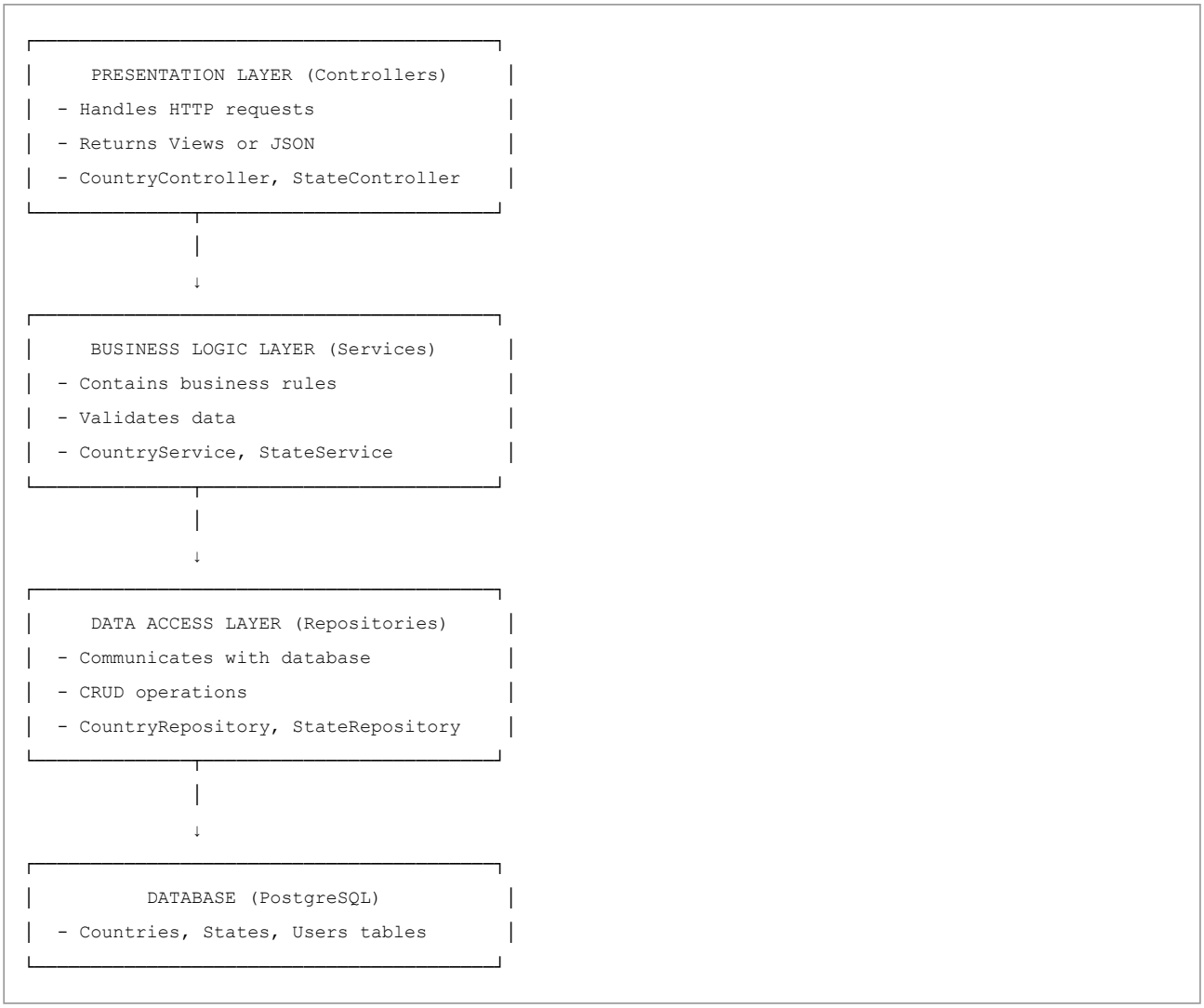
1. **Three-Layer Architecture** - Separation of concerns
2. **AJAX-based CRUD** - No page reloads
3. **Cascading Dropdowns** - State dropdown changes based on Country selection
4. **Soft Delete** - Records are hidden, not permanently deleted
5. **Dependency Injection** - Loose coupling between components

---

Architecture & Design Patterns

Three-Layer Architecture

This project follows a **three-layer architecture** pattern:



Why Three Layers?

Benefits:

1. **Separation of Concerns** - Each layer has a specific responsibility
2. **Maintainability** - Easy to modify one layer without affecting others
3. **Testability** - Each layer can be tested independently
4. **Reusability** - Business logic can be reused across different controllers

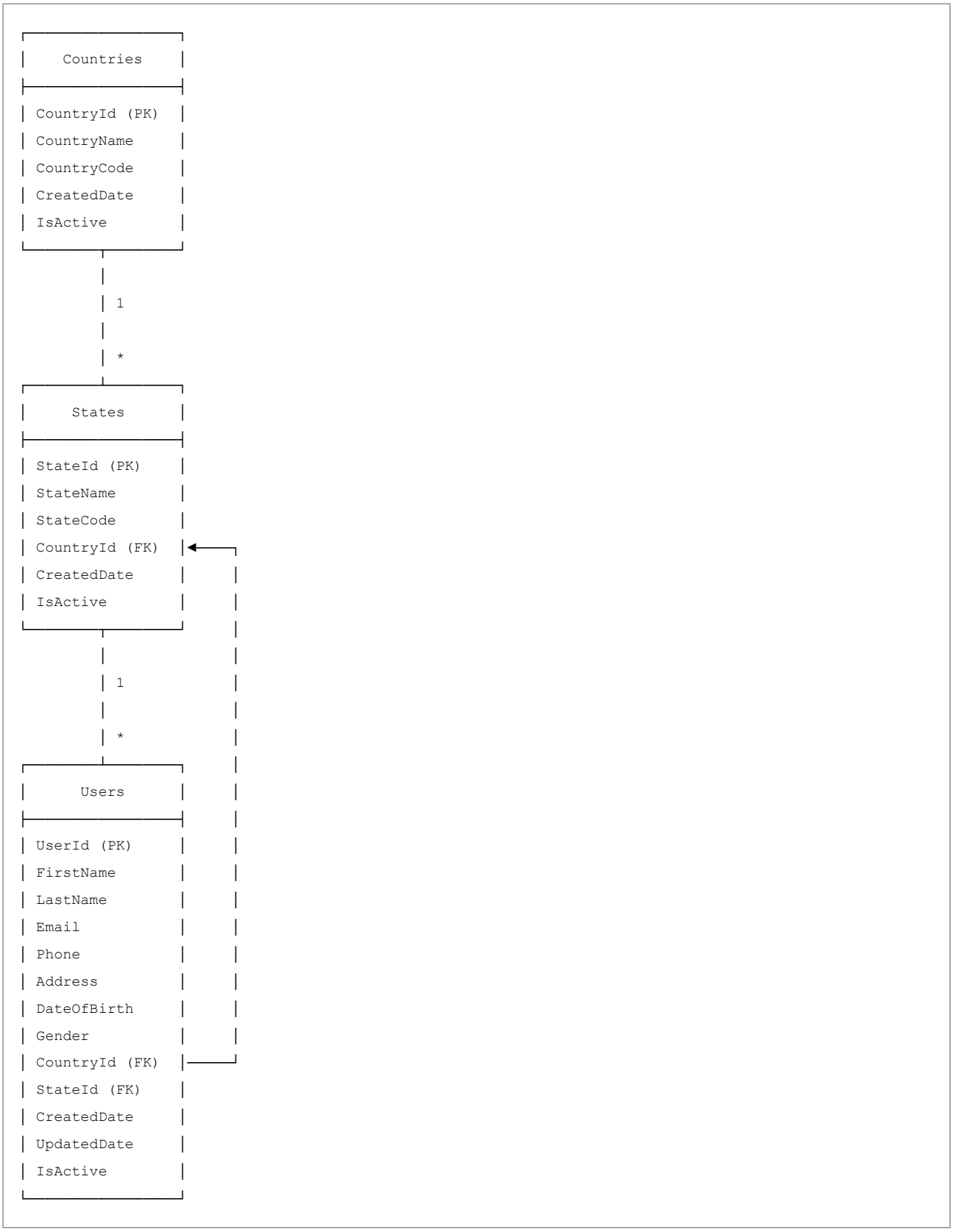
---

Project Structure

```
Practicecrudwithajax/
├── Model/                          # Data Models (Entities)
│   ├── CountryModel/
│   │   └── Country.cs              # Country entity
│   ├── StateModel/
│   │   └── State.cs                # State entity
│   └── UserModel/
│       └── User.cs                 # User entity
│
├── Data/                           # Database Context
│   ├── ApplicationDbContext.cs     # EF Core DbContext
│   └── Migrations/                 # Database migrations
│
├── Dal/                            # Data Access Layer (Repositories)
│   ├── CountryDAL/
│   │   ├── ICountryRepository.cs # Interface
│   │   └── CountryRepository.cs  # Implementation
│   ├── StateDAL/
│   │   ├── IStateRepository.cs
│   │   └── StateRepository.cs
│   └── UserDAL/
│       ├── IUserRepository.cs
│       └── UserRepository.cs
│
├── Bal/                            # Business Logic Layer (Services)
│   ├── CountryBAL/
│   │   ├── ICountryService.cs    # Interface
│   │   └── CountryService.cs     # Implementation
│   ├── StateBAL/
│   │   ├── IStateService.cs
│   │   └── StateService.cs
│   └── UserBAL/
│       ├── IUserService.cs
│       └── UserService.cs
│
└── Practicecrudwithajax/           # Web Application
    ├── Controllers/                # MVC Controllers
    │   ├── CountryController.cs
    │   ├── StateController.cs
    │   └── UserController.cs
    ├── Views/                      # Razor Views
    │   ├── Country/
    │   │   └── Index.cshtml
    │   ├── State/
    │   │   └── Index.cshtml
    │   └── User/
    │       └── Index.cshtml
    ├── wwwroot/                    # Static files (CSS, JS, images)
    ├── Program.cs                  # Application entry point
    └── appsettings.json            # Configuration
```

Database Design

Entity Relationship Diagram (ERD)



Relationships

1. **Country** → **States** (One-to-Many)
    - One Country can have many States
    - Each State belongs to one Country
2. **Country** → **Users** (One-to-Many)
    - One Country can have many Users
    - Each User belongs to one Country
3. **State** → **Users** (One-to-Many)
    - One State can have many Users
    - Each User belongs to one State

Soft Delete Strategy

Instead of permanently deleting records, we use **soft delete**:

- Each table has an `IsActive` field (boolean)
- When "deleting", we set `IsActive = false`
- Queries filter by `IsActive = true` to show only active records

Benefits:

- Data recovery is possible
- Maintains referential integrity
- Audit trail preserved

Layer-by-Layer Explanation

1. Model Layer (Entities)

Purpose: Define the structure of data

Example: Country.cs

```
public class Country
{
    public int CountryId { get; set; }          // Primary Key
    public string CountryName { get; set; }     // Required field
    public string? CountryCode { get; set; }    // Optional field
    public DateTime CreatedDate { get; set; }
    public bool IsActive { get; set; }

    // Navigation Properties (Relationships)
    public virtual ICollection<State>? States { get; set; }
    public virtual ICollection<User>? Users { get; set; }
}
```

Key Concepts:

- **Data Annotations** - `[Required]`, `[StringLength]`, etc. for validation
- **Navigation Properties** - Define relationships between entities

- **Virtual keyword** - Enables lazy loading in Entity Framework

2. Data Layer (ApplicationDbContext)

Purpose: Bridge between application and database

Example: ApplicationDbContext.cs

```
public class ApplicationDbContext : DbContext
{
    public DbSet<Country> Countries { get; set; }
    public DbSet<State> States { get; set; }
    public DbSet<User> Users { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Configure relationships
        modelBuilder.Entity<Country>()
            .HasMany(c => c.States)
            .WithOne(s => s.Country)
            .HasForeignKey(s => s.CountryId);

        // Global query filter for soft delete
        modelBuilder.Entity<Country>()
            .HasQueryFilter(c => c.IsActive);
    }
}
```

Key Concepts:

- **DbSet** - Represents a table in the database
- **OnModelCreating** - Configure entity relationships and constraints
- **HasQueryFilter** - Automatically filters soft-deleted records

3. Repository Layer (Data Access)

Purpose: Perform database operations (CRUD)

Example: CountryRepository.cs

```
public class CountryRepository : ICountryRepository
{
    private readonly ApplicationDbContext _context;

    public CountryRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<List<Country>> GetAllCountriesAsync()
    {
        return await _context.Countries
            .Where(c => c.IsActive)
            .OrderBy(c => c.CountryName)
            .ToListAsync();
    }

    public async Task<Country> AddCountryAsync(Country country)
    {
        country.CreatedDate = DateTime.UtcNow;
        country.IsActive = true;
        await _context.Countries.AddAsync(country);
        await _context.SaveChangesAsync();
        return country;
    }

    // ... other CRUD methods
}
```

Key Concepts:

- **LINQ Queries** - `.Where()`, `.OrderBy()`, `.FirstOrDefaultAsync()`
- **Async/Await** - Non-blocking database operations
- **SaveChangesAsync()** - Commits changes to database

4. Service Layer (Business Logic)

Purpose: Business rules and validation

Example: CountryService.cs

```
public class CountryService : ICountryService
{
    private readonly ICountryRepository _repository;

    public CountryService(ICountryRepository repository)
    {
        _repository = repository;
    }

    public async Task<List<Country>> GetAllCountriesAsync()
    {
        // Add business logic here if needed
        // Example: Log the operation, check permissions, etc.
        return await _repository.GetAllCountriesAsync();
    }

    public async Task<Country> AddCountryAsync(Country country)
    {
        // Business validation
        // Example: Check if country name already exists
        return await _repository.AddCountryAsync(country);
    }
}
```

Key Concepts:

- **Business Validation** - Rules specific to your application
- **Abstraction** - Service doesn't know about database details
- **Single Responsibility** - Each service handles one entity

5. Controller Layer (Presentation)

Purpose: Handle HTTP requests and return responses

Example: CountryController.cs

```
public class CountryController : Controller
{
    private readonly ICountryService _countryService;

    public CountryController(ICountryService countryService)
    {
        _countryService = countryService;
    }

    // Returns the view
    public IActionResult Index()
    {
        return View();
    }

    // AJAX endpoint - Returns JSON
    [HttpGet]
    public async Task<JsonResult> GetAllCountries()
    {
        var countries = await _countryService.GetAllCountriesAsync();
        return Json(countries);
    }

    // AJAX endpoint - Creates country
    [HttpPost]
    public async Task<JsonResult> Create(Country country)
    {
        if (ModelState.IsValid)
        {
            await _countryService.AddCountryAsync(country);
            return Json(new { success = true, message = "Country added!" });
        }
        return Json(new { success = false, message = "Invalid data" });
    }
}
```

Key Concepts:

- **Action Methods** - Methods that handle requests
- **HTTP Verbs** - `[HttpGet]`, `[HttpPost]`
- **JsonResult** - Returns JSON for AJAX calls
- **ModelState** - Validation state of the model

Controllers Explained

Why Do We Need Controllers?

Controllers are the **entry point** for all HTTP requests in an MVC application.

Flow:

1. User clicks a button or submits a form
2. Browser sends HTTP request to server
3. **Controller** receives the request
4. Controller calls **Service** for business logic
5. Service calls **Repository** for data
6. Data flows back: Repository → Service → Controller
7. Controller returns **View** or **JSON** to browser

CountryController - Detailed Explanation

Purpose: Manage Country CRUD operations

Methods:

1. **Index()** - Returns the main view

```
public IActionResult Index()
{
    return View();  // Returns Views/Country/Index.cshtml
}
```

2. **GetAllCountries()** - AJAX endpoint to fetch all countries

```
[HttpGet]
public async Task<JsonResult> GetAllCountries()
{
    var countries = await _countryService.GetAllCountriesAsync();
    return Json(countries);
}
```

- **Why?** To populate the countries table on the page
- **When called?** When page loads or after add/update/delete
- **Returns:** JSON array of countries

3. **Create(Country country)** - AJAX endpoint to add country

```
[HttpPost]
public async Task<JsonResult> Create(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.AddCountryAsync(country);
        return Json(new { success = true, message = "Country added!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

- **Why?** To add a new country to database
- **When called?** When user submits the "Add Country" form
- **Returns:** JSON with success status and message

4. **GetById(int id)** - AJAX endpoint to fetch single country

```
[HttpGet]
public async Task<JsonResult> GetById(int id)
{
    var country = await _countryService.GetCountryByIdAsync(id);
    return Json(country);
}
```

- **Why?** To populate the edit form with existing data
- **When called?** When user clicks "Edit" button
- **Returns:** JSON object of the country

5. **Update(Country country)** - AJAX endpoint to update country

```
[HttpPost]
public async Task<JsonResult> Update(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.UpdateCountryAsync(country);
        return Json(new { success = true, message = "Country updated!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

- **Why?** To save changes to an existing country
- **When called?** When user submits the "Edit Country" form
- **Returns:** JSON with success status and message

6. **Delete(int id)** - AJAX endpoint to delete country

```
[HttpPost]
public async Task<JsonResult> Delete(int id)
{
    var result = await _countryService.DeleteCountryAsync(id);
    if (result)
    {
        return Json(new { success = true, message = "Country deleted!" });
    }
    return Json(new { success = false, message = "Error deleting" });
}
```

- **Why?** To soft delete a country (set IsActive = false)
- **When called?** When user clicks "Delete" button and confirms
- **Returns:** JSON with success status and message

StateController - Special Features

Cascading Dropdown Implementation:

```
[HttpGet]
public async Task<JsonResult> GetStatesByCountry(int countryId)
{
    var states = await _stateService.GetStatesByCountryIdAsync(countryId);
    var result = states.Select(s => new
    {
        s.StateId,
        s.StateName
    });
    return Json(result);
}
```

Why is this important?

- When user selects a Country, the State dropdown should show only states from that country
- This method filters states by CountryId
- Called via AJAX when Country dropdown changes

Example Flow:

1. User selects "India" from Country dropdown
2. JavaScript detects the change
3. AJAX call to `/State/GetStatesByCountry?countryId=1`
4. Returns only Indian states
5. State dropdown is populated with Indian states

UserController - Complete Registration

Purpose: Handle user registration with Country and State selection

Key Method:

```
[HttpPost]
public async Task<JsonResult> Create(User user)
{
    if (ModelState.IsValid)
    {
        await _userService.AddUserAsync(user);
        return Json(new { success = true, message = "User registered!" });
    }

    var errors = ModelState.Values
        .SelectMany(v => v.Errors)
        .Select(e => e.ErrorMessage);
    return Json(new { success = false, message = "Validation failed", errors });
}
```

Validation:

- Email format validation
- Required field validation
- Phone number format validation
- All defined in User model with Data Annotations

AJAX Implementation

What is AJAX?

AJAX = Asynchronous JavaScript and XML

Benefits:

- Update parts of a page without reloading
- Better user experience
- Faster interactions

AJAX Flow in This Project

```
User Action (Click button)
    ↓
JavaScript Event Handler
    ↓
$.ajax() call to server
    ↓
Controller receives request
    ↓
Service processes request
    ↓
Repository accesses database
    ↓
Data returns to Controller
    ↓
Controller returns JSON
    ↓
AJAX success callback
    ↓
Update page with new data
```

Example: Add Country with AJAX

Frontend (JavaScript):

```
function addCountry() {
    var countryData = {
        CountryName: $('#countryName').val(),
        CountryCode: $('#countryCode').val()
    };

    $.ajax({
        url: '/Country/Create',
        type: 'POST',
        data: countryData,
        success: function(response) {
            if (response.success) {
                alert(response.message);
                loadCountries();  // Refresh the table
                $('#addForm').trigger('reset');  // Clear form
            } else {
                alert(response.message);
            }
        },
        error: function() {
            alert('Error occurred');
        }
    });
}
```

Backend (Controller):

```
[HttpPost]
public async Task<JsonResult> Create(Country country)
{
    if (ModelState.IsValid)
    {
        await _countryService.AddCountryAsync(country);
        return Json(new { success = true, message = "Country added!" });
    }
    return Json(new { success = false, message = "Invalid data" });
}
```

Key Points:

1. JavaScript collects form data
2. AJAX sends POST request to `/Country/Create`
3. Controller validates and saves data
4. Controller returns JSON response
5. JavaScript receives response and updates UI

Setup & Installation

Prerequisites

1. **.NET 8.0 SDK** - Download from microsoft.com/dotnet
2. **PostgreSQL** - Download from postgresql.org
3. **Visual Studio 2022** or **VS Code**
4. **Git** (optional)

Step-by-Step Setup

Step 1: Clone or Download Project

```
git clone <repository-url>
cd Practicecrudwithajax
```

Step 2: Configure Database Connection

Edit `appsettings.json`:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Database=PracticeCrudDb;Username=postgres;Password=yourpassword"
  }
}
```

Replace:

- `localhost` - Your PostgreSQL server address
- `PracticeCrudDb` - Your database name
- `postgres` - Your PostgreSQL username
- `yourpassword` - Your PostgreSQL password

Step 3: Restore NuGet Packages

```
dotnet restore
```

Step 4: Create Database

The database will be created automatically when you run migrations.

Step 5: Apply Migrations

```
cd Data
dotnet ef database update
```

This creates all tables in your PostgreSQL database.

Step 6: Run the Application

```
cd ../Practicecrudwithajax
dotnet run
```

Step 7: Open in Browser

```
https://localhost:5001
```

Troubleshooting

Problem: "Connection refused" error **Solution:** Make sure PostgreSQL is running

Problem: "Migration not found" **Solution:** Run `dotnet ef migrations add InitialCreate` first

Problem: "Port already in use" **Solution:** Change port in `launchSettings.json`

API Endpoints Reference

Country Endpoints

Method	Endpoint	Purpose	Parameters	Returns
GET	`/Country/Index`	Display page	None	HTML View
GET	`/Country/GetAllCountries`	Get all countries	None	JSON array
GET	`/Country/GetById?id={id}`	Get single country	id (int)	JSON object
POST	`/Country/Create`	Add country	Country object	JSON response
POST	`/Country/Update`	Update country	Country object	JSON response
POST	`/Country/Delete?id={id}`	Delete country	id (int)	JSON response

State Endpoints

Method	Endpoint	Purpose	Parameters	Returns
GET	`/State/Index`	Display page	None	HTML View
GET	`/State/GetAllStates`	Get all states	None	JSON array
GET	`/State/GetStatesByCountry?countryId={id}`	Get states by country	countryId (int)	JSON array
GET	`/State/GetById?id={id}`	Get single state	id (int)	JSON object
POST	`/State/Create`	Add state	State object	JSON response
POST	`/State/Update`	Update state	State object	JSON response
POST	`/State/Delete?id={id}`	Delete state	id (int)	JSON response

User Endpoints

Method	Endpoint	Purpose	Parameters	Returns
GET	`/User/Index`	Display page	None	HTML View
GET	`/User/GetAllUsers`	Get all users	None	JSON array
GET	`/User/GetById?id={id}`	Get single user	id (int)	JSON object
POST	`/User/Create`	Register user	User object	JSON response
POST	`/User/Update`	Update user	User object	JSON response
POST	`/User/Delete?id={id}`	Delete user	id (int)	JSON response

Common Concepts for Beginners

1. What is Dependency Injection?

Simple Explanation: Instead of creating objects yourself, you ask the framework to provide them.

Without DI:

```
public class CountryController
{
    private ICountryService _service;

    public CountryController()
    {
        _service = new CountryService();  // Creating manually
    }
}
```

With DI:

```
public class CountryController
{
    private ICountryService _service;

    public CountryController(ICountryService service)
    {
        _service = service;  // Framework provides it
    }
}
```

Benefits:

- Easier to test (can inject mock objects)
- Loose coupling (controller doesn't know about implementation)
- Centralized configuration (in Program.cs)

2. What is Async/Await?

Purpose: Don't block the application while waiting for database

Synchronous (Blocking):

```
public List<Country> GetAllCountries()
{
    // Application waits here, can't do anything else
    return _context.Countries.ToList();
}
```

Asynchronous (Non-blocking):

```
public async Task<List<Country>> GetAllCountriesAsync()
{
    // Application can handle other requests while waiting
    return await _context.Countries.ToListAsync();
}
```

Benefits:

- Better performance

- Can handle more concurrent users
- Application remains responsive

3. What is LINQ?

LINQ = Language Integrated Query

Purpose: Query collections using C# syntax

Example:

```
// Get all active countries, sorted by name
var countries = _context.Countries
    .Where(c => c.IsActive)           // Filter
    .OrderBy(c => c.CountryName)      // Sort
    .ToList();                        // Execute

// This generates SQL:
// SELECT * FROM Countries WHERE IsActive = true ORDER BY CountryName
```

Common LINQ Methods:

- `.Where()` - Filter
- `.OrderBy()` - Sort ascending
- `.OrderByDescending()` - Sort descending
- `.Select()` - Transform/project
- `.FirstOrDefault()` - Get first or null
- `.Any()` - Check if any exists
- `.Count()` - Count records

4. What is Entity Framework Core?

EF Core = Object-Relational Mapper (ORM)

Purpose: Work with database using C# objects instead of SQL

Without EF Core (Raw SQL):

```
string sql = "INSERT INTO Countries (CountryName, IsActive) VALUES (@name, @active)";
// Execute SQL command...
```

With EF Core:

```
var country = new Country { CountryName = "India", IsActive = true };
_context.Countries.Add(country);
_context.SaveChanges();
```

Benefits:

- Type-safe (compiler checks for errors)
- Less code
- Database-agnostic (can switch databases easily)
- Automatic SQL generation

5. What is Model Validation?

Purpose: Ensure data meets requirements before saving

Data Annotations:

```
public class Country
{
    [Required(ErrorMessage = "Country name is required")]
    [StringLength(100, ErrorMessage = "Max 100 characters")]
    public string CountryName { get; set; }

    [EmailAddress(ErrorMessage = "Invalid email format")]
    public string Email { get; set; }
}
```

Validation in Controller:

```
if (ModelState.IsValid)
{
    // Data is valid, save it
}
else
{
    // Data is invalid, return errors
}
```

6. What is Soft Delete?

Hard Delete: Permanently remove record from database

```
DELETE FROM Countries WHERE CountryId = 1
```

Soft Delete: Mark record as inactive

```
UPDATE Countries SET IsActive = false WHERE CountryId = 1
```

Benefits:

- Can recover deleted data
- Maintains referential integrity
- Audit trail preserved

Implementation:

```
public async Task<bool> DeleteCountryAsync(int id)
{
    var country = await _context.Countries.FindAsync(id);
    if (country == null) return false;

    country.IsActive = false;  // Soft delete
    await _context.SaveChangesAsync();
    return true;
}
```

7. What is Cascading Dropdown?

Purpose: Second dropdown depends on first dropdown selection

Example:

1. User selects "India" from Country dropdown
2. State dropdown shows only Indian states (Maharashtra, Delhi, etc.)
3. User selects "USA" from Country dropdown
4. State dropdown changes to show only US states (California, Texas, etc.)

Implementation:

```
$('#countryDropdown').change(function() {
    var countryId = $(this).val();

    $.ajax({
        url: '/State/GetStatesByCountry',
        data: { countryId: countryId },
        success: function(states) {
            $('#stateDropdown').empty();
            $.each(states, function(i, state) {
                $('#stateDropdown').append(
                    $('<option>').val(state.stateId).text(state.stateName)
                );
            });
        }
    });
});
```

Conclusion

This documentation covers all aspects of the ASP.NET Core CRUD application with AJAX. The project demonstrates:

1. **Clean Architecture** - Three-layer separation
2. **Best Practices** - Dependency Injection, Async/Await, Repository Pattern
3. **Modern Web Development** - AJAX, RESTful APIs, Responsive UI
4. **Database Design** - Relationships, Soft Delete, Migrations

Next Steps for Learning

1. **Add Authentication** - User login/logout
2. **Add Authorization** - Role-based access control
3. **Add Logging** - Track errors and operations
4. **Add Unit Tests** - Test your code
5. **Add API Documentation** - Swagger/OpenAPI
6. **Deploy to Cloud** - Azure, AWS, or other platforms

Resources

- **Official Docs:** https://docs.microsoft.com/aspnet/core (https://docs.microsoft.com/aspnet/core)
- **Entity Framework:** https://docs.microsoft.com/ef/core (https://docs.microsoft.com/ef/core)
- **C# Guide:** https://docs.microsoft.com/dotnet/csharp (https://docs.microsoft.com/dotnet/csharp)
- **PostgreSQL:** https://www.postgresql.org/docs (https://www.postgresql.org/docs)

Document Version: 1.0

Last Updated: January 2026

Author: Practice CRUD with AJAX Project

GitHub
GIT CHEAT SHEET

Git is the free and open source distributed version control system that's responsible for everything GitHub related that happens locally on your computer. This cheat sheet features the most important and commonly used Git commands for easy reference.

INSTALLATION & GUIS

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows
https://windows.github.com

GitHub for Mac
https://mac.github.com

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms
http://git-scm.com

SETUP

Configuring user information used across all local repositories

`git config --global user.name "[firstname lastname]"`
set a name that is identifiable for credit when review version history
`git config --global user.email "[valid-email]"`
set an email address that will be associated with each history marker
`git config --global color.ui auto`
set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

`git init`
initialize an existing directory as a Git repository
`git clone [url]`
retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

`git status`
show modified files in working directory, staged for your next commit
`git add [file]`
add a file as it looks now to your next commit (stage)
`git reset [file]`
unstage a file while retaining the changes in working directory
`git diff`
diff of what is changed but not staged
`git diff --staged`
diff of what is staged but not yet committed
`git commit -m "[descriptive message]"`
commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

`git branch`
list your branches. a * will appear next to the currently active branch
`git branch [branch-name]`
create a new branch at the current commit
`git checkout`
switch to another branch and check it out into your working directory
`git merge [branch]`
merge the specified branch's history into the current one
`git log`
show all commits in the current branch's history

INSPECT & COMPARE

Examining logs, diffs and object information

`git log`	show the commit history for the currently active branch
`git log branchB..branchA`	show the commits on branchA that are not on branchB
`git log --follow [file]`	show the commits that changed file, even across renames
`git diff branchB...branchA`	show the diff of what is in branchA that is not in branchB
`git show [SHA]`	show any object in Git in human-readable format

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

`git remote add [alias] [url]`	add a git URL as an alias
`git fetch [alias]`	fetch down all the branches from that Git remote
`git merge [alias]/[branch]`	merge a remote branch into your current branch to bring it up to date
`git push [alias] [branch]`	Transmit local branch commits to the remote repository branch
`git pull`	fetch and merge any commits from the tracking remote branch

TRACKING PATH CHANGES

Versioning file removes and path changes

`git rm [file]`	delete the file from project and stage the removal for commit
`git mv [existing-path] [new-path]`	change an existing file path and stage the move
`git log --stat -M`	show all commit logs with indication of any paths that moved

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

`git rebase [branch]`	apply any commits of current branch ahead of specified one
`git reset --hard [commit]`	clear staging area, rewrite working tree from specified commit

IGNORING PATTERNS

Preventing unintentional staging or commiting of files

```
logs/
*.notes
pattern*/
```

Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.

`git config --global core.excludesfile [file]`	system wide ignore pattern for all local repositories

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

`git stash`	Save modified and staged changes
`git stash list`	list stack-order of stashed file changes
`git stash pop`	write working from top of stash stack
`git stash drop`	discard the changes from top of stash stack